



Отличие ассистента от агента

Когда мы говорим об ИИ-ассистенте, я имею в виду систему, которая в первую очередь отвечает на наш запрос. Мы формулируем задачу, передаём ей текст, код или описание проблемы, а она возвращает объяснение, предложение, фрагмент кода или последовательность действий. После этого следующий шаг обычно выполняет человек: копирует исправленный код, создаёт файл, запускает тесты, проверяет результат и снова обращается к ассистенту, если что-то не получилось. Даже если такой ассистент подключён к редактору через плагин и видит открытые файлы, основной режим взаимодействия всё равно строится вокруг подсказки: человек спрашивает, система отвечает, а решение о выполнении действия остаётся за человеком.

Агент работает иначе. Для него запрос может быть не отдельным вопросом, а целью, которую нужно довести до результата. Получив задачу, агент способен сам определить необходимые шаги, посмотреть доступные файлы, внести изменения, выполнить проверку, увидеть ошибку, скорректировать решение и повторить действие. Именно поэтому здесь появляются планирование и итерации. Если после первого изменения тесты не проходят, работа агента не обязательно заканчивается на сообщении «тест не прошёл». Он может проанализировать причину, изменить решение и снова выполнить проверку. Поэтому главное различие заключается не в том, насколько хорошо система умеет писать код. Один и тот же базовый интеллектуальный механизм может использоваться в обоих случаях. Различие заключается в уровне действия: ассистент в основном предлагает, а агент способен выполнять последовательность операций над рабочей средой и проверять результат.

для преподавателя далее используются нейронки. Отправляйте одно и то же и в обычный интерфейс чата и в чат ИИ-агента.

Сценарий 1. Автофикс

Для демонстрации достаточно одного файла `calculator.py`.

```
def divide(a, b):  
    return a / b  
  
def average(numbers):  
    return sum(numbers) / len(numbers)
```

Здесь есть две ситуации, которые могут приводить к ошибкам во время выполнения: деление на ноль и вычисление среднего значения пустого списка.

Промпт:

Найди проблемы, которые могут приводить к ошибкам во время выполнения, исправь их и проверь получившееся решение.

При работе в обычном диалоге мы, скорее всего, получим описание проблем и предложенный вариант исправленного кода. В агентном режиме задача может продолжиться непосредственно в рабочей среде: система изучит файл, внесёт изменения и попытается проверить результат.

Сценарий 2. Автотесты

Создаём файл `discount.py`.

```
def calculate_discount(price, percent):  
    if percent < 0 or percent > 100:  
        raise ValueError("Некорректная скидка")  
  
    return price - price * percent / 100
```

Функция принимает стоимость товара и процент скидки. Допустимым считается процент от 0 до 100 включительно. При значении вне этого диапазона должна возникать ошибка.

Промпт:

Подготовь полноценную проверку функции `calculate_discount` для основных, граничных и ошибочных случаев и убедись, что проверки проходят.

Здесь особенно хорошо видна разница между генерацией ответа и выполнением задачи. Ассистент может написать подходящие тесты в сообщении. Агент при наличии рабочей среды способен самостоятельно определить, куда добавить тестовый файл, создать проверки, запустить их и использовать результат запуска как следующий источник информации.

Сценарий 3. Автодокументация

Создаём файл `users.py`.

```
def create_user(name, email):  
    if not name:  
        raise ValueError("Имя обязательно")  
  
    if "@" not in email:  
        raise ValueError("Некорректный email")  
  
    return {"name": name, "email": email}  
  
def find_user(users, email):  
    for user in users:  
        if user["email"] == email:  
            return user
```

```
return None
```

Дополнительной документации для этого небольшого модуля изначально нет.

Промпт:

Подготовь документацию для этого модуля так, чтобы новый разработчик мог понять его назначение, функции, параметры, возвращаемые значения, возможные ошибки и пример использования.

В режиме ассистента результатом обычно становится готовый текст документации, который ещё нужно самостоятельно перенести в проект. В агентном режиме появляется возможность превратить эту же цель в действие над рабочими материалами: проанализировать модуль и оформить соответствующую документацию непосредственно рядом с кодом.

Сценарий 4. Автомиграция

Для этого примера создаём два файла.

Файл `app.py`:

```
from flask import Flask

app = Flask(__name__)

@app.route("/hello/<name>")
def hello(name):
    return {"message": f"Привет, {name}"}
```

Файл `requirements.txt`:

```
Flask
```

Смысл исходного приложения очень простой: существует HTTP-маршрут `/hello/<name>`, который принимает имя из адреса и возвращает JSON с приветствием.

Промпт:

Перенеси это приложение с Flask на FastAPI, сохрани существующее поведение и приведи необходимые зависимости в соответствие с новой реализацией.

Здесь отличие становится ещё заметнее. В обычном диалоге система может показать, каким должен стать файл приложения, и отдельно объяснить, как изменить зависимости. Это уже полезный результат, но изменения остаются предложением. Агент получает цель, которая затрагивает несколько файлов. Он может определить, какие из них необходимо изменить, выполнить миграцию согласованно и затем проверить состояние проекта.

Во всех четырёх экспериментах формулировка задачи намеренно не содержит указаний вроде «только объясни», «покажи код», «отредактируй файл» или «запусти тест». Мы задаём конечную цель. Благодаря этому различие проявляется естественно: диалоговый ассистент обычно возвращает результат в форме ответа, тогда как агентная среда может интерпретировать ту же формулировку как работу, которую необходимо последовательно выполнить и проверить.
